# The AI Coding Governance Kit

Boris Cherny Framework for Claude Code Skills and Codex CLI Commands

By Max Dion - CEO, AXKI
AI Automation Consultancy - Montreal, Canada
axki.ca | max@axki.ca

Version 1.0 - April 2026

**This kit ships separate Claude Code skills and Codex CLI commands. The governance concepts are the same, but the invocation syntax is intentionally different to avoid copy-paste confusion.**

# Table of Contents


| # | Section | Page |
|---|---|---|
| 1 | Why AI Coding Governance Matters | 3 |
| 2 | The Boris Cherny Framework | 4 |
| 3 | Public Invocation Names | 5 |
| 4 | Framework 1: Quick Checklist | 6 |
| 5 | Framework 2: Plan Validator | 7 |
| 6 | Framework 3: Adversarial Watchdog | 9 |
| 7 | Role-Agnostic Architecture | 11 |
| 8 | Installation Guide | 12 |
| 9 | Real-World Validation | 13 |
| 10 | Ready to Implement This? | 14 |


This PDF matches the public Claude Code skill names and Codex command names shipped in the repository.

# 1. Why AI Coding Governance Matters

AI coding assistants are powerful, but without governance they become liability generators. Left unchecked, AI agents hallucinate file references, claim work is done when it is not, bundle unrelated changes into monolithic commits, skip testing, and push unverified code toward production.

The result is technical debt that compounds quickly: phantom features, tests that pass because they test nothing, and a codebase that looks productive on the surface but fails under scrutiny.

Boris Cherny's framework solves this by imposing a strict phase-gate protocol on AI coding sessions. Every phase requires explicit human approval before proceeding. Every commit must be atomic. Every claim must be verifiable. A dedicated adversarial reviewer catches what the builder misses.

## Common AI Coding Failures

| Failure mode | Impact | Governance response |
|---|---|---|
| Hallucinated file references | Broken builds and phantom features | Watchdog hallucination checks |
| Skipped planning phase | Scope creep and wasted cycles | HITL gates before implementation |
| Bundled commits | Impossible rollbacks | Atomic commit rule |
| Tests that test air | False confidence | Test validity review |
| No memory updates | Context loss between sessions | Memory discipline checks |
| Self-validated work | Confirmation bias | Separate reviewer role |

# 2. The Boris Cherny Framework

The Boris Cherny framework enforces a strict sequential loop for AI-assisted development. Each phase has a clear entry gate, defined deliverables, and requires explicit human approval before the next phase can begin.

| Phase | Purpose | Gate | Deliverable |
|---|---|---|---|
| EXPLORE | Understand the problem space | GATE 1 -> GO | Problem definition and constraints |
| PLAN | Design the solution | GATE 2 -> GO | Approved implementation plan |
| IMPLEMENT | Write code in scoped lanes | GATE 3 -> GO | Tested code with evidence |
| COMMIT | Atomic commits and authorized release | GATE 4 -> GO Prod | Clean history and approved push |

## Core Principles

- Sequential phases: Explore -> Plan -> Implement -> Commit. Never skip.
- Human-in-the-loop gates: explicit GO required between phases.
- Atomic commits: one concern per commit. No bundled unrelated changes.
- Lane worktrees are conditional: if the project uses them, create real git worktrees in the project-approved lanes directory; otherwise follow the project's AGENTS.md/CLAUDE.md workflow.
- Stop-hooks: define test commands and loop until all green. No premature success claims.
- Memory discipline: update governance files such as CLAUDE.md or AGENTS.md when the project requires it.
- Adversarial review: a dedicated reviewer audits the executor before merge or release.

**Project-specific AGENTS.md / CLAUDE.md instructions override this generic framework.**

# 3. Public Invocation Names

Keep Claude Code skills and Codex commands distinct. Do not rename them to generic slash commands.

| Claude Code Skills | Codex Commands |
|---|---|
| /skill:axki-init | /codex:axki-init |
| /skill:axki-checklist | /codex:axki-checklist |
| /skill:axki-validate | /codex:axki-validate |
| /skill:axki-watchdog | /codex:axki-watchdog |

The document uses both columns throughout the framework so readers can copy the correct invocation for the tool they are using. Claude Code users copy the skill names. Codex CLI users copy the command names.

## Session Role Initialization

```
/skill:axki-init executor=codex reviewer=claude
/codex:axki-init executor=codex reviewer=claude
```

The executor writes code, creates commits, runs tests, and updates project memory. The reviewer validates plans, runs checklists, deploys the watchdog, and issues GO / NEEDS REVISION / BLOCK verdicts.

# 4. Framework 1: Quick Checklist

### Claude Code skill: /skill:axki-checklist

### Codex command: /codex:axki-checklist

A rapid governance audit to run before any commit or phase transition. The reviewer scores each item against the executor's work using PASS / FAIL / PARTIAL.

### PHASE & LOOP

[ ] Phase identified? (Explore/Plan/Implement/Commit)
[ ] Loop order respected? No skipping.
[ ] Plan Mode active before any writes?
[ ] Explicit GO received before this phase started?

### LANES & WORKTREES

[ ] If the project uses lane worktrees, real Git worktrees were used - not folders?
[ ] If the project uses lane worktrees, they were created in the project-approved lanes directory?
[ ] Each lane has one concern only?
[ ] No cross-lane contamination?
[ ] Worktrees removed after each lane merges?

### GATES

[ ] GATE 1 cleared? (Explore -> GO)
[ ] GATE 2 cleared? (Plan -> GO)
[ ] GATE 3 pending? (Implement -> GO before Commit)
[ ] GATE 4 active? (No prod without GO Prod)

### DONE CRITERIA

[ ] test_cmd defined?
[ ] done_when defined per task?
[ ] Stop-hook loops until all green?
[ ] Failure protocol: fix -> new commit -> rerun?

### COMMITS / MEMORY / ANTI-PATTERNS

[ ] Atomic? One concern per commit?
[ ] Memory file (CLAUDE.md / AGENTS.md) updated this session?
[ ] Approved target branch/current session branch pushed only when authorized?
[ ] No phantom file references?
[ ] No tests that test air?
[ ] No LLM output assumed correct?
[ ] No over-engineering beyond current scope?

```
Fails found: ___
Partials found: ___

VERDICT: GO / NEEDS REVISION / BLOCK
```

# 5. Framework 2: Plan Validator

### Claude Code skill: /skill:axki-validate

### Codex command: /codex:axki-validate

Validate a proposed plan against the full Boris Cherny protocol before the executor begins implementation. Run this before any GATE 2 approval.

### LOOP & PHASE

[ ] Current phase clearly identified?
[ ] Order respected? No Implement without approved Plan.
[ ] Plan Mode active? No file writes before GO.
[ ] One clear GO gate before the next phase?

### LANES & WORKTREES

[ ] If the project uses lane worktrees, real Git worktrees are planned with explicit git worktree add commands?
[ ] If the project uses lane worktrees, create real git worktrees in the project-approved lanes directory; otherwise follow the project's AGENTS.md/CLAUDE.md workflow.
[ ] Each lane named and assigned before Implement?
[ ] Each lane scoped to one concern?
[ ] Worktrees created one at a time?
[ ] Removal after each merge planned?

### HITL GATES

[ ] GATE 1 present? (Explore complete -> wait for GO)
[ ] GATE 2 present? (Plan approved -> wait for GO)
[ ] GATE 3 defined? (Implement done -> GO before Commit)
[ ] GATE 4 respected? (No push to prod without GO Prod)

### STOP-HOOKS / COMMITS / REVIEW

[ ] Done-when defined before Implement starts?
[ ] Stop-hook command specified per task?
[ ] Will agent loop until all green?
[ ] Failure protocol: fix -> new commit -> rerun?
[ ] Monotonic test count enforced?
[ ] Atomic commits planned?
[ ] Reviewer agent defined in workflow?
[ ] Final verification after session planned?

## MEMORY

[ ] Memory file (CLAUDE.md / AGENTS.md) update planned at session end?
[ ] Known-errors memory update planned if the project uses one?
[ ] Session log planned if the project requires one?
[ ] Push the approved target branch/current session branch only when authorized?
[ ] Merged branches deletion planned?

## ANTI-PATTERNS CHECK

[ ] No monolithic scripts?
[ ] No skipped Plan Mode?
[ ] No prod push without GO Prod?
[ ] No LLM output assumed correct?
[ ] No over-engineering beyond current scope?
[ ] No phantom file references?

## SECURITY (if OAuth or external API)

[ ] State validation planned?
[ ] Public paths exempted for callbacks?
[ ] CSRF exemption for webhooks/callbacks?
[ ] New users assigned to correct tenant only?
[ ] JWT parity with existing auth flow?

```
Fails found: ___
Partials found: ___

VERDICT: GO / NEEDS REVISION / BLOCK
```

# 6. Framework 3: Adversarial Watchdog

### Claude Code skill: /skill:axki-watchdog

### Codex command: /codex:axki-watchdog

The most aggressive audit tool. The reviewer assumes the executor is trying to look productive and systematically proves or disproves that claim. This is not a code review; it is a lie-detection protocol for AI agents.
Reviewer mindset: Senior adversarial watchdog. Catch lies, gaps, hallucinations, and protocol violations. You are not a builder. Do not suggest code. Verify claims independently.

### CHALLENGE A - FILE & MEMORY DISCIPLINE

[ ] Did executor actually write to memory file (CLAUDE.md / AGENTS.md)?
[ ] Did executor update current-state memory if the project uses one?
[ ] Did executor update known-errors memory if the project uses one?
[ ] Did executor write output to correct files with confirmation read?
[ ] Did executor push only the approved target branch/current session branch when authorized?
[ ] Did executor write session log if the project requires one?

### CHALLENGE B - TASK PLAN COMPLIANCE

[ ] List every task from the approved plan.
[ ] For each task: DONE / PARTIAL / NOT DONE / CLAIMED BUT UNVERIFIED.
[ ] Flag any task marked done without visible proof.
[ ] Flag scope creep: work not in approved plan.
[ ] Flag silent drops: plan items with no explanation.

### CHALLENGE C - TEST VALIDITY

[ ] Did the tests actually run? Show terminal output.
[ ] Are test assertions meaningful?
[ ] Did any test fail? Zero failures on complex implementation is suspicious.
[ ] Is test count monotonically increasing?

## CHALLENGE D - HALLUCINATION DETECTION

[ ] Did executor reference a file never shown?
[ ] Did executor claim a feature works without real execution trace?
[ ] Did executor say this is implemented for any known gap?
[ ] Did executor describe future work as current work?

## CHALLENGE E - BORIS CHERNY PROTOCOL

[ ] Did executor exit Plan Mode before GO?
[ ] Were commits atomic?
[ ] Did any lane work happen outside assigned worktree?
[ ] If the project uses lane worktrees, were real git worktrees created in the project-approved lanes directory?
[ ] Were merged worktrees removed?
[ ] Were merged branches deleted?
[ ] Was GATE 4 respected?

## FINAL WATCHDOG REPORT

```
MEMORY & FILE DISCIPLINE: [VERIFIED / PARTIAL / FAILED]
TASK PLAN COMPLIANCE: [VERIFIED / PARTIAL / FAILED]
TEST VALIDITY: [REAL / THEATER / UNVERIFIED]
HALLUCINATIONS DETECTED: [YES - list / NONE FOUND]
BC PROTOCOL VIOLATIONS: [list each / NONE FOUND]
UNVERIFIED CLAIMS: [list each / NONE FOUND]
PHANTOM FILES: [list each / NONE FOUND]
SUSPICIOUS ASSERTIONS: [list each / NONE FOUND]
TASKS CLAIMED DONE UNVERIFIED: [list each / NONE FOUND]
SCOPE CREEP: [list each / NONE FOUND]

FINAL VERDICT: GO / NEEDS REVISION / BLOCK
REQUIRED BEFORE NEXT SESSION: [actions / NONE]
```

## Optional paste block only when a project asks for executor-ready remediation text:

```
---
PASTE THIS INTO EXECUTOR:
[short direct instructions]
---
```

# 7. Role-Agnostic Architecture

The AI landscape shifts constantly. Hardcoding model names into governance frameworks creates brittle systems. AXKI governance uses roles instead: executor and reviewer.
The user declares who builds and who validates at session start. The frameworks reference roles, not tool brands. Swap freely without reinstalling.

| Role | Responsibilities | Uses |
|---|---|---|
| EXECUTOR (Builder) | Write code, create commits, run tests, manage project-approved lanes, update memory files. | Receives reviewer verdicts |
| REVIEWER (Validator) | Validate plans, run checklists, deploy watchdog, issue GO / NEEDS REVISION / BLOCK verdicts, catch hallucinations. | /skill:axki-checklist or /codex:axki-checklist /skill:axki-validate or /codex:axki-validate /skill:axki-watchdog or /codex:axki-watchdog |

## Examples

```
/skill:axki-init executor=claude reviewer=codex
/codex:axki-init executor=codex reviewer=claude
```

When a new model drops, change the executor/reviewer values. Keep Claude Code invocations on /skill:axki-* and Codex CLI invocations on /codex:axki-*.

# 8. Installation Guide

## For Claude Code (Skills Format)

Claude Code uses a .claude/skills/ directory in your project root. Each AXKI skill lives in its own directory with a SKILL.md file.

```
git clone https://github.com/AXKI-Consulting/axki-governance-public.git
cp -r axki-governance-public/.claude/ /path/to/your/project/.claude/
```

### Available Claude Code skills after install:

```
/skill:axki-init
/skill:axki-checklist
/skill:axki-validate
/skill:axki-watchdog
```

## For Codex CLI (AGENTS.md + Commands Format)

Codex CLI uses an AGENTS.md file in your project root with command files copied into the project's command directory.

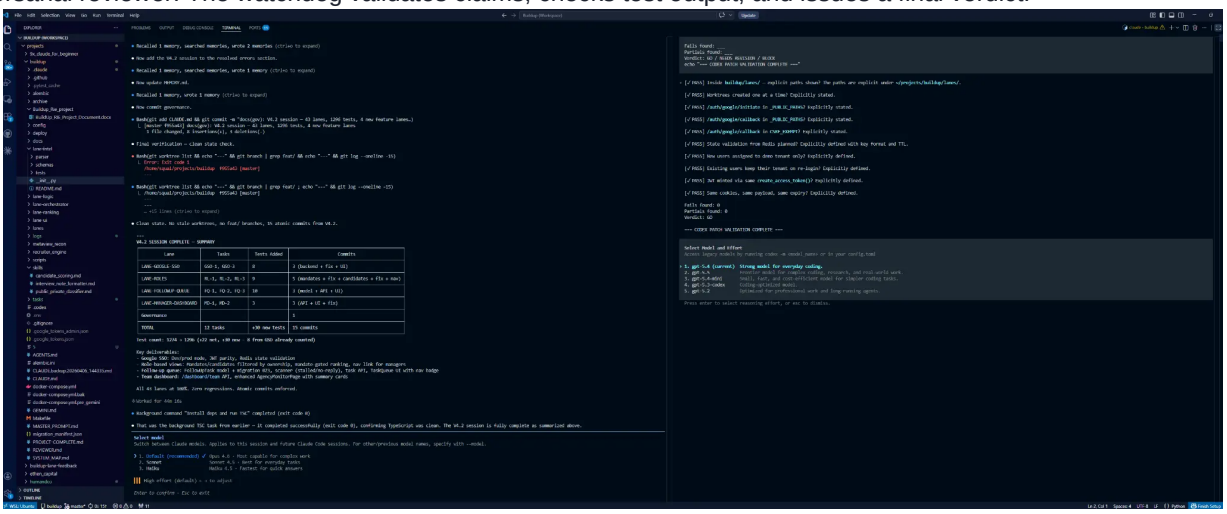
```
git clone https://github.com/AXKI-Consulting/axki-governance-public.git
cp axki-governance-public/codex/AGENTS.md /path/to/your/project/AGENTS.md
cp -r axki-governance-public/codex/commands/ /path/to/your/project/.codex/commands/
```

### Available Codex commands after install:

```
/codex:axki-init
/codex:axki-checklist
/codex:axki-validate
/codex:axki-watchdog
```

# 9. Real-World Validation

Below is a real screenshot from a production-style session where one AI acts as executor and the other acts as adversarial reviewer. The watchdog validates claims, checks test output, and issues a final verdict.



Governance works when every PASS has evidence, every FAIL has an explanation, and the final verdict is earned rather than assumed.

## Release Checklist for This Kit

[ ] Claude Code skill names use only /skill:axki-*.
[ ] Codex command names use only /codex:axki-*.
[ ] No generic /axki:* syntax remains.
[ ] No hardcoded default-branch push rule remains.
[ ] Lane worktree guidance is conditional and project-approved.
[ ] Watchdog output includes unverifiable claims, phantom files, suspicious assertions, scope creep, final verdict, and required next-session actions.

# 10. Ready to Implement This?

These frameworks are free and open-source. Implementing AI governance across a real engineering team - with custom lanes, project-specific stop-hooks, and multi-agent workflows - still requires operational judgment.
AXKI helps engineering teams deploy structured AI coding governance so they ship faster without losing control. The framework can be adapted to your stack, repository workflow, and release process.

## Book a Free Discovery Call with Max

Let's discuss how to implement Boris Cherny governance in your AI-assisted development workflow.

axki.ca | max@axki.ca
AXKI - AI Automation Consultancy | Montreal, Canada